

A Novel Framework for Enterprise Web Services Change Management



M. Thirumaran
Assistant Professor
Department of Computer Science and
Engineering
Pondicherry Engineering College,
Puducherry, INDIA

Dhavachelvan Ponnuramam
Professor and Head
Department of Computer Science
School of Engineering
Pondicherry University
Puducherry, INDIA

NagaVenkataKiran. G, Divya. A
PG Scholars (M.Tech-CSE)
Department of Computer Science and
Engineering
Pondicherry Engineering College
Puducherry, INDIA

ABSTRACT

Business logic might be constantly improved or changed in order to provide enhanced services or to reduce cost. As a result, the software applications employed must be continuously updated to comply with the changes in their functional features. Changes in business logic of the web service must be done very quickly within the given time constraint as the services consumed by the providers and clients must not be affected. The framework proposed in this paper responds these challenges in three fold: Firstly, it formats the request and discovers required service logic through rule manager. Secondly, extracted logics are built as a complete web service and FSM is constructed by the Business Logic model to state the logic flow. Thirdly, it ascertains the right way for evaluation by analyzing dependency between the service logics through FSM. This allows enterprises to carve up and assimilate the services dynamically without developer's intervention at any stage. Thus this can be used for any IT enterprises in modern service industry to reduce the threshold of development and operation of services.

Categories and Subject Descriptors

H.4 [Information Systems Applications]: Office Automation – Workflow management

General Terms

Management, Measurement, Documentation, Performance, Design, Economics, Reliability, Experimentation, Human Factors, Standardization, Languages, Theory, Legal Aspects, and Verification.

Keywords

Web Services, Change Management, Business Logic Management, Finite State Machine, Code Generation, Run time management, Workflow systems.

1. INTRODUCTION

The Business Logic Management System (BLMS) is modular. Each module can be easily integrated with existing modules so that the applications can acquire added functionality as the need arises. Those modules provide built-in support for the

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

ICACCI'12, August 3-5, 2012, Chennai, T Nadu, India.

Copyright 2012 ACM 978-1-4503-1196-0/12/08...\$10.00.

core requirements of any advanced business system which includes the ability to automate decision making using a rule-based process, a monitoring engine that lets us configure any number of check points, support for integral workflow and BPM (Business Process Management) capabilities and the ability to communicate with other systems. BLMS transparently communicates with the information consumers want without forcing those consumers to know anything about the underlying technology of the BLMS including the language it was written in, the version of the compiler used, or the type and structure of any associated databases. It is capable of supporting web services and other current technologies like HTML, XML, SOAP, etc. The IT Department does not have to hold out an excessive learning when implementing the BLMS. They have to build a complete documentation and actual code examples to make the transition to a logic-based system easier and to accelerate the implementation and deployment process. Thus, the goal of implementing BLMS is to allow the organization to ultimately reduce the cost by automating tasks that used to require large human intervention.

It is important to react to change faster than other competitors since the reaction time of the changes to occur are decreasing day by day. The blending of change and reaction time is setting even more pressure on information technology teams as time elapses. BLMS serves it. BLMS has schemes to exteriorize business rules and to provide a facility for centralized business rule management. This addresses an urgent need existing in this business era to change their business rules in order to adapt to a rapidly changing business environment, and to overcome the confining nature of slow IT change cycles. It also allows managing and writing rules in a familiar business language and hence the business users can update software according to their schedule, rather than IT's. When the business logic which includes the policies and knowledge of the business are abstracted from the application logic the rule-based application becomes successful by making the process of creating and changing business logic much easier. The backbone of any Business Logic Management System is the business rule which can be defined in simple words as a statement that constrains or governs some aspect of the business. Business rules are atomic and cannot be decomposed into more detailed business rules. In order to make changes to the business rule in run time the business rule engines are used. Rule engine is commonly provided as a component of a business rule management system which, among other functions, provides the ability to register, define, classify, and manage all the rules, verify consistency of rule definitions and define the relationships between different rules and combines them together to draw conclusions.

It offers a rule repository which is a centralized location where the entire business logic is stored. The purpose of a

business rules repository is to support the business rule information needs of all the stakeholders in a business rules-based approach to the initial development of systems and their lifetime enhancement. Specifically, the purpose of a business rules repository is to provide the ability to audit the rules and define multiple versions of rules. This system also has rule templates which are decision tables or patterns that are structural prototypes with the same structure but different rules that might be applicable to various data. Rule templates allow rule designers to write the rules logic once and use it many times with the help of empty slots that can be filled in later. With rule templates we can completely hide rules implementation details from the business users. In order to check the syntax of a rule, first these rules must have a formal representation. The symbol system which is easiest for human to learn is hardest for a computer to master. Hence an appropriate rule engine which has all the above functionalities has to be selected. Many rules engines are available that includes both commercial and open source choices. Commercial rules engines express rules in English-like language. Others open source versions write rules using scripting languages such as Groovy or Python. The rule engine which we select should contain the minimal requirements which permit business analysts to create and alter the rules, employs a fully-featured repository, and lets applications to be deployed in service oriented architecture.

2. A FRAMEWORK FOR WEB SERVICES CHANGE MANAGEMENT

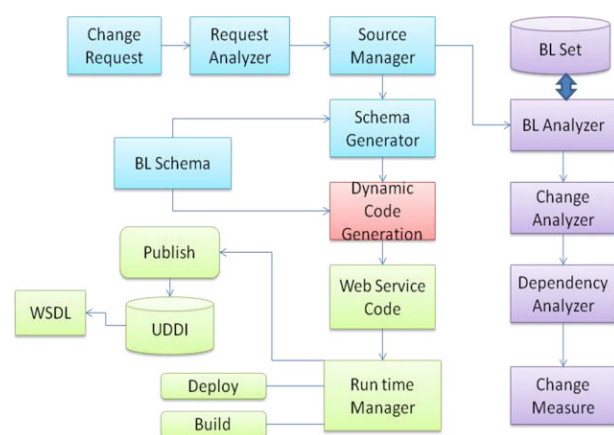


Figure 1. Framework for Web Services Change Management.

The framework for web services change management is illustrated in figure 1. Here, the Business Logic Analyzer is mainly to analyze the given request and to ascertain necessary rules and functions to process the requisite. Context analyzer here discovers the environment which the given request correlated to. It grasps the request, slices it into numerous parts, scrutinizes each part as a separate keyword and ascertains the domain which the detained request related to. In the ascertained domain it traces required business logics to process the request. If it is the first time for logic analyzer to access the predicted business logic, it dissects the logic into business rule, function and domain variable set. Then it builds up Business Logic schema through schema generator as its tag categorizes rules, functions and parameter set with unique ID for each. BL schema depicts the logic flow obviously. Then it put asides isolated rules/function in the rule/function repository with its unique ID. Then it uncovers

required rules and function set in the business logic. Next time while accessing the logic, Logic Analyzer recognizes the required business rules and functions of the isolated logic directly from rule/function repository. Rule repository comprises business rules of each and every service by isolating it with unique ID. The repository also contains service name with its ID for each sustained rule. Through which source manager locates business rule in the business logic. Prime functions of this component are providing authentication to access the code, monitoring changes in the source code and notifying violation of rules.

Dependency Analyzer discovers dependencies with the ascertained rules and functions in the business logic through the identified change criteria. The presence of this FSM simulator is for the automation of the business logic set for its abstraction and completeness. For constructing conceptual model for the business logic, Turing Machine(TM) computational model has been incorporated and the dynamic behavior of the logic change is observed with the help of state transition table (STT) constructed from the transition function with respect to the change scenario.

Change Analyzer is to let authorized one to do meaningful changes, monitor and manage the changes. It maintains a change log which records the change progress with completeness and exceptions with respect to the change request. It initially decides whether the change can be handled at run time. The proposed model guides business analyst and other management people to do modification at BL schema level by considering all the run time issues and also business policy constraints. The actual execution of the change leads to the creation and/or alteration of code and when this change is propagated it probably causes other code fragments to change as well. Change Analyzer scrutinizes the validity of modified rules, functions and parameters through TM simulator. If the transition of TM ends at some stage, it decides changes are feasible and commits the modified logic otherwise it roll backs and suggests the necessary solution with respect to the change context. After the initial test results seem satisfactory, the performance measure can be brought up to date and be released, together with the committed changes. Finally, the source manager accepts the valid change and automatically builds and deploys the service within the reaction time and closes the change request entry in the change log. Moreover Change Analyzer investigates the defects in the logic and validates the quality of business logic by automating various functional and non-functional assessments to ensure that new changes have not impaired existing functionality. In association with the dependency analyzer it assess the business logic through methods such as Order of execution, Time complexity, Schema Validation, Mapping function and Similarity measure. In that, the order of execution ensures the work flow of the business logic along with the consistent updating of the logic sequence with respect to the changes. The Time complexity measure is used to predict the approximate complexity nature of the changes. Schema validation supports the changes at the schema level by providing the code abstraction to the business parties to easily understand the service logic with the help of business vocabularies. The mapping function ensures the policy constraints associated with the logic and also capable of mapping reducibility factors between the rules and functions within the business set. The Similarity measure evaluates the changes with respect to the source construct with functionality. Schema Generator is a chief component in the model whose core work is to generate and uphold BL schema at various stage. At first phase, after logic analyzer sorts out the

business logic in to rule, function and domain variable set, it generates BL schema as its tags catalogs rule, function and domain variable. Change manager uses this BL schema to guide management people who are not familiar in code to made fruitful changes. The changes made in the schema will be updated obviously in business logic. Same way, when programmers do some changes in logic, it gets updated in schema accurately. Run time manager is responsible for building and deploying the services after fruitful changes are done and also it helps to debug the exceptions in the service logic sophistically.

3. CASE STUDY

Travel agency enterprise architecture was used to implement the system proposed to show the benefits of it. Various services constitute the architecture and each is coordinated with each other to provide maximum benefits to customers as all the functionalities are integrated in it as shown in figure 2. The run-time view of the system is designed and the key is given for the entire process work flow.

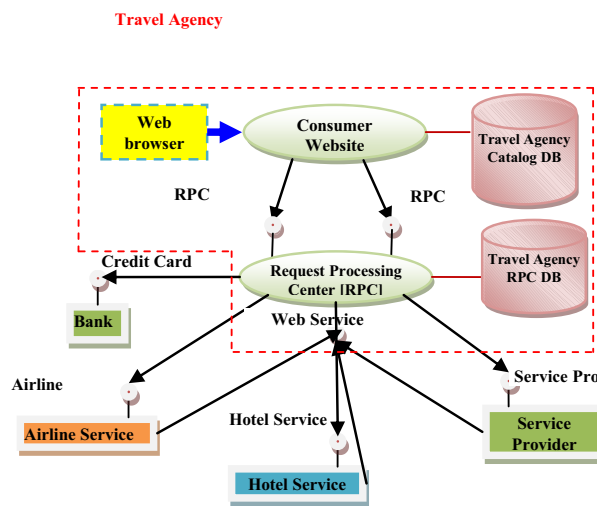
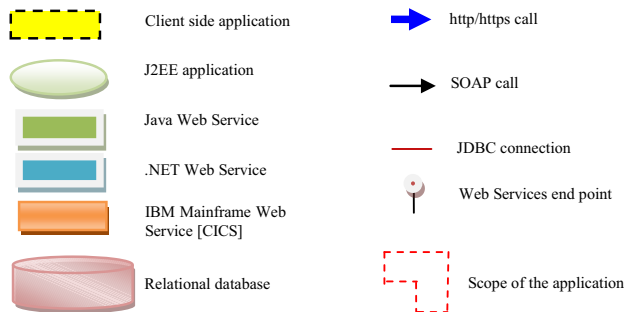


Figure 2. Run-time view of the Travel Agency Web Services based Architecture.

Key:



The business logic schema of Airline_Reservation_Service before change is shown below:

```
<?xml version="1.0" encoding="utf-8" ?>
<AmountValidation>
  <Rule function="AvailableTicket" >
    <paratype="int" name="AvailableTickets,">
    </para>
    <para type="int" name="TicketsBook">
    </para>
    <if
      condition="AvailableTickets<TicketsBo
      ok">
      <process>
        <ouput_statement>
          <print>"Ticket Available"
          </print>
        </ouput_statement>
      </process>
    </if>
    <else>
      <process>
        <output_statement>
          <print>"Ticket_Not_Available"
          </print>
        </output_statement>
      </process>
    </else>
  </Rule>
</AmountValidation>
```

The business logic schema of Airline_Reservation_Service after changes induced is shown below:

```
<?xml version="1.0" encoding="utf-8" ?>
<AmountValidation>
  <Rule function="AvailableTicket" >
    <para type="int" name="AvailableTickets
    ,">
    </para>
    <para type="int" name="TicketsBook">
    </para>
    <if
      condition="AvailableTickets<TicketsBo
      ok">
      <process>
        <output_statement>
          <print>"Ticket Available"
          </print>
        </output_statement>
      </process>
    </if>
    <else>
      <process>
        <output_statement>
          <print>"Ticket Not Available"
          </print>
        </output_statement>
      </process>
    </else>
  </Rule>
```

```

<Rule function="AddConcession" >
  <para type="int" name="Age ," >
  </para>
  <para type="int" name="TicketsBook" >
  </para>
  <if condition="Age>65">
    <process>
      <output_statement>
        <print>"Concession Available"
      </print>
      </output_statement>
    </process>
  </if>
  <else>
    <process>
      <output_statement>
        <print>"No Concession"
      </print>
      </output_statement>
    </process>
    <return>Yes
  </return>
  </else>
</Rule>
</AmountValidation>

```

Now the changes done at the schema level are mapped to the corresponding source code without the intervention of source code developer. All the changes are done at business analyst side.

The business logic schema can be even extended to process the SQL commands such as DML statements (Select, Insert, Delete, Modify) as shown below:

```

<?xml version="1.0" encoding="utf-8" ?>
<SqlQuerys>
  <Select>
    <query type="DML" name="select"
    table="Vechicle">
      <rowset condition=
"Vechicle_NoSheet>1">
        <columns>

<column_name>Vechicle_name</column_name>

<column_name>Vechicle_NoSheet</column_name>

        </columns>
      </rowset>
    </query>
  </Select>
  <Update>
    <query type="DML" name="update"
    table="Vechicle" >
      <rowset condition="VechicleID=1">
        <columns>
          <column_name
assignment="Vechicle_name='name'"></column_name>
        </columns>
      </rowset>
    </query>
  </Update>

```

```

<Insert>
  <query type="DML" name="insert"
  table="Vechicle">
    <rowset >
      <columns>
        <column_name
value="kiran">Vechicle_name</column_name>
      </column_name>
        <column_name
value="12">Vechicle_NoSheet</column_name>
      </column_name>
      </columns>
    </rowset>
  </query>
</Insert>
<Delete>
  <query type="DML" name="Delete"
  table="Vechicle">
    <rowset condition=
"Vechicle_NoSheet=10">
      </rowset>
    </query>
  </Delete>
</SqlQuerys>

```

4. IMPLEMENTATION

We have successfully completed the implementation of the system newly devised using ASP.NET 3.5 using web services technology and C# coding. All the services were built and run time changes are introduced at the business logic of the web service source code through business logic schema written in the form of XML statements. The snapshots of the prototype implemented are shown as follows:

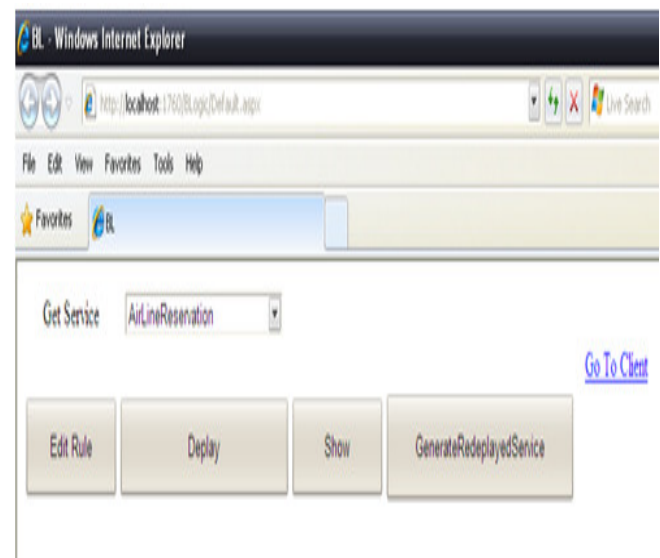


Figure 3. User Interface to Business Analyst.

In figure 3, graphical user interface (GUI) is shown where the business analyst can get the services available and the process of change management can be implemented at the business logic of the web service.

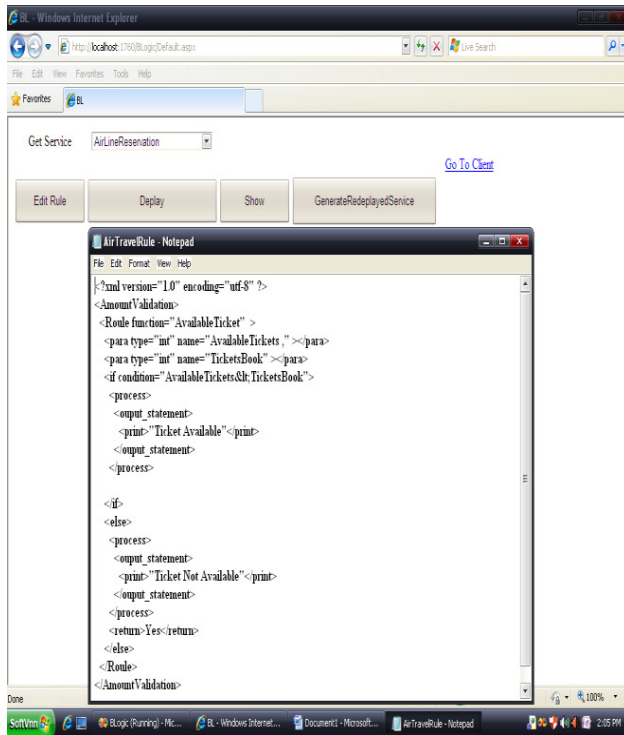


Figure 4. Airline reservation business logic schema loaded for incorporating changes.

In figure 4, business logic schema is loaded for airline reservation where changes can be done such as addition, modification or deletion of business rules is possible.

Table 1. Evaluation of Change Management

Business Services	Change Scrutinizing Time (ms)	Business Logic Origination Time (ms)	Business Logic Generation Time (ms)	Change Implementation Time (ms)
Airline	0.22	0.22	0.33	0.23
Credit Card	0.23	0.53	0.23	0.45
Taxi	0.32	0.32	0.32	0.22
Lodging	0.21	0.45	0.33	0.23
Vehicle	0.13	0.33	0.23	0.15
Hotel	0.22	0.32	0.32	0.22

The above table gives the consequences of performance measures for various services implemented in our case study. They are identified with respect to change scrutinizing time, business logic origination time, business logic generation time, and change implementation time. Also the total time is calculated for analyzing the time taken to redeploy the services after changes are done at business logic level. The performance measure for the above case study is given in the following graph depicted as Figure 5.

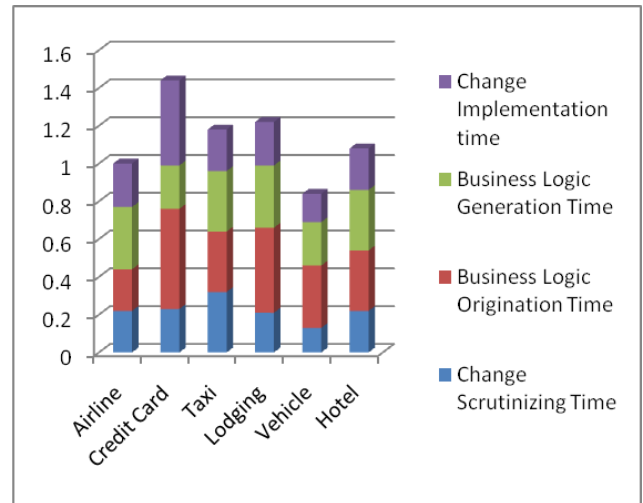


Figure 5. Performance evaluation of various services implemented with respect to time

5. EVALUATION METRICS FOR CHANGE FACTORS

The entire result containing the evaluation of change measure for the given change request and their corresponding change measure in the rule/function/parameter level are tabulated in Table 1.

6. RELATED WORKS

Yang et al presented a method for detecting failures in implementation of services that are running currently in concurrent order and also from rules in Service Oriented architecture based atmospheres. It also finds the dependencies that co exist among the services and are extracted from log files when any change occurs [1]. Business dealings are obligatory to accomplish in the company of changes within legislation and strategy. However the successful executions of innovative legislation are a lot expensive, might include long escort period which cause huge loss to industry trade. Yiwei Gong and others proposed a novel technique for business processes in favor of building liveness and agility while applying changes through innovative or improvised policies. It also elaborated a new research method for software reuse by joint policy execution [2]. Dimitris Apostolou et al presented an ontology driven method for managing changes in e-Government services based on SOA. It enables methodical reaction of these systems to changes by applying proper techniques intended for accomplishing stability whenever a change is exposed [3]. In [4], the authors presented a novel method to represent changes in ontologies which have been organized in layers and provided data independence from versions, etc. Also deposits of change circulation stratagems are allowed to keep distributed copy of the same ontology synchronized. A methodology for handling changes in Long-term Composed Services (LCS) was proposed by Xumin Liu, Athman Bouguettaya and others. In this technique the authors have categorized changes in two ways:- Top-down and Bottom-Up scaffolds. Also the functional and non-functional properties of the composed services have been identified [5], [6], [14], [16], [20], [21], and [22]. Hans Weigand et al presented an elucidation to policy changes in business atmospheres by dividing the SOA based architectures on policy enforcements and their specifications [7]. Their method has been highly successful to

Adaptive Service Oriented Architectures (ASOA). H. Yahyaoui and others formulated a method to manage and handle the changes propagated at run time through Web Service Policy Language (WSPL) which permits perspective pattern by mutually policy and rule stages [8]. Pedro Antunes and Hernâni Mourão throw in Business Process Management systems (BPM) through resilience sustain. The system introduced a deposit of services integrating resilience support in BPM. Harry Jiannan Wang and J. Leon Zhao modeled a new approach for changes in work flow management systems called Constraint-centric Workflow Change Analytics (CWCA). The core functionality of this system is it analyzes change anomalies in workflow based architectures. An algorithm (procedure) is used to discover changes in SOA context and open-source rule engine is exposed to afford a verification of notion execution of CWCA [10]. Shari S.C. Shang, Eldon Y. Li, Ya-Ling Wu, Oliver C.L. Hou have done a survey on existing life cycle models in context of web services. These representations illustrate the variety of open Web 2.0 relevance and presented a scaffold in support of a superior kind of working patterns and significance scheme in the Web 2.0 model [11]. Minhong Wang and Huaqing Wang presented a model for managing business logic of the services within the business processes. Since many of the existing models cope up with changes at process level, this paper has identified the changes propagated at business layer [12]. Daniel Żmuda, Marek Psiuk and Krzysztof Zieliński presented some innovative challenges in SOA based environments to observe and watch the implementation strategies occurring dynamically. Dynamic run-time reconfiguration is the prime concept of that approach [13].

7. CONCLUSION

The proposed model which has been implemented provides good benefits to the business analysts who want to implement changes at the business logic layer of the software process (addition, deletion and modification of business logic) by themselves without the help of technical staff and hence reduces the implementation costs. The change management process can be done very easily and provides much flexibility. First, the analyst gets the services of the enterprise web service which are available and business logic schema file is loaded for the service logic where he wants to do modifications. After doing changes, the corresponding source code is generated for the changes done in the BL Schema. And finally the web service is re-deployed automatically for the changes done in the service code through BL Schema. At the client side, the modified services can be accessed and can be included into the software process.

8. REFERENCES

- [1] Yang Xiao, Urban, S., "Using Rules and Data Dependencies for the Recovery of Concurrent Processes in a Service-Oriented Environment", IEEE Transactions on Services Computing, Issue 1, Volume 5, 2012, pp.59-71.
- [2] Yiwei Gong, Marijn Janssen, "From policy implementation to business process management: Principles for creating flexibility and agility", Government Information Quarterly, Volume 29, Supplement 1, Elsevier B.V, January 2012, Pages S61–S71.
- [3] Dimitris Apostoloua, Gregoris Mentzasb, Ljiljana Stojanovicc, Barbara Thoenssend, Tomás Pariente Lobo, "A collaborative decision framework for managing changes in e-Government services", Government Information Quarterly Volume, Elsevier B.V., January 2011, Pages 101–116.
- [4] Raúl Palmaa, Oscar Corchoa, Asunción Gómez-Pérez, Peter Haase, "A holistic approach to collaborative ontology development based on change management", Elsevier journal of Web Semantics: Science, Services and Agents on the World Wide Web, Volume 9, Issue 3, September 2011, Pages 299–314.
- [5] Xumin Liu, Athman Bouguettaya, Jemma Wu, and Li Zhou, "Ev-LCS: A System for the Evolution of Long-term Composed Services", IEEE Transactions on Services Computing, Issue: 99, ISSN : 1939-1374, 23 June 2011.
- [6] Xumin Liu, Athman Bouguettaya, Qi Yu, Zaki Malik, "Efficient change management in long-term composed services", ORIGINAL RESEARCH PAPER, Journal of Service Oriented Computing and Applications, Springer-Verlag London Limited 2010, pp.87-103.
- [7] Hans Weigand, Willem-Jan van den Heuvel, Marcel Hiel, "Business policy compliance in service-oriented systems", Elsevier journal of Information Systems, Volume 36, Issue 4, June 2011, Pages 791–807.
- [8] H. Yahyaoui, L. Wang, A. Mourad, M. Almulah, Q.Z. Sheng, "Towards context-adaptable Web service policies", in Procedia Computer Science, Volume 5, 2011, Pages 610–617.
- [9] Pedro Antunesa, Hernâni Mourão, "Resilient Business Process Management: Framework and services", Elsevier journal of Expert Systems with Applications, Volume 38, Issue 2, February 2011, Pages 1241–1254.
- [10] Harry Jiannan Wang, J. Leon Zhao, "Constraint-centric workflow change analytics", Elsevier journal of Decision Support Systems, Volume 51, Issue 3, June 2011, Pages 562–575.
- [11] Shari S.C. Shang, Eldon Y. Li, Ya-Ling Wu, Oliver C.L. Hou, "Understanding Web 2.0 service models: A knowledge-creating perspective", Elsevier journal of Information & Management, Volume 48, Issues 4–5, May 2011, Pages 178–184.
- [12] Minhong Wang, Huaqing Wang, "From process logic to business logic—A cognitive approach to business process management", Elsevier journal of Information & Management, Volume 43, Issue 2, March 2006, Pages 179–193.
- [13] Daniel Żmuda, Marek Psiuk, Krzysztof Zieliński, "Dynamic monitoring framework for the SOA execution environment", in proceedings of ICCS 2010, Procedia Computer Science, Volume 1, Issue 1, May 2010, Pages 125–133.
- [14] Salman Akram, Athman Bouguettaya, Xumin Liu, Armin Haller, Florian Rosenberg, Xiaobing Wu. A Change Management Framework for Service Oriented Enterprises, International Journal of Next-Generation Computing (IJNGC), Vol. 1, No. 1, 09 2010, Pages 1-077.
- [15] T. Erl, Service-Oriented Architecture: Concepts, Technology, and Design, Prentice Hall PTR, Upper Saddle River, NJ, USA (2005).
- [16] Xumin, Akram, Salman, Bouguettaya, Athman, Change Management for Semantic Web Services, 1st Edition., ISBN 978-1-4419-9328-1, 2011.
- [17] Ontology Support for Managing Top-down Changes in Composite Services. X. Liu, A. Bouguettaya, The 4th International Conference on Collaborative Computing (CollaborateCom 2008), Orlando, FL, Nov. 13-16. 2008.

Table 1. Evaluation Metrics

Change request			Change Factor	Primitive Changes	Dependency Relation (D)	Change Measure
Rule Set	Function	Parameters				
(Add) (delete) (modify)	(Add) (delete) (modify)	(Add) (delete) (modify)	Execution Order	Adding a new rule to the existing execution order	Based on FSM transition function between rules	$\frac{\partial R}{\partial o} = d(R)[(1/\omega) + (1/F) - (c/n)]$
(R ₁ ,R ₂,R _n)	F ₁₁ , F ₁₂F _α	P ₁₁₁ ,P ₁₁₂,P _β		Deleting a rule from the execution order relation		
				Modifying the existing order with few changes		
(R ₁ ,R ₂,R _n)	F ₁₁ , F ₁₂F _α	P ₁₁₁ ,P ₁₁₂,P _β	Time complexity	Addition of a rule of new time complexity	Rules related to same time complexity on similar input set	$\Delta T = (\frac{1}{kb+1}m^{kb+1} - 1) - (\frac{1}{b+1}m^{b+1} - 1)$
				Deleting a rule of some Time complexity		
				Modification of time complexity of existing one		
(R ₁ ,R ₂,R _n)	F ₁₁ , F ₁₂F _α	P ₁₁₁ ,P ₁₁₂,P _β	Schema	Adding a rule of new schema varying from existing one	Rules of similar schema	$\Delta S_{ce} = S_{ce} - S_c$ $\Delta S_{cs(e)} = S_{cs(e)} - S_c$ $\Delta S_{c\mu} = S_{c\mu} - S_c$
				Deletion of a part or whole schema		
				Modifying the existing schema		
(R ₁ ,R ₂,R _n)	F ₁₁ , F ₁₂F _α	P ₁₁₁ ,P ₁₁₂,P _β	Mapping Function	Adding a rule based on the mapping function	Based on similar mapping between rules and functions including policy constraints	$\Delta F = f(L,R,F) - f_c(L,R,F')$ $\Delta R = f(L,R,F) - f_c(L,R',F)$ $\Delta L = f(L,R,F) - f_c(L',R,F)$
				Deleting a rule from the mapping dependency		
				Modification of the existing rule in order to map the dependency		
(R ₁ ,R ₂,R _n)	F ₁₁ , F ₁₂F _α	P ₁₁₁ ,P ₁₁₂,P _β	Similarity Measure	Addition of a new construct to the existing one	Based on the analysis of constructs implemented in the rule	$\frac{\partial S_c}{\partial x_{p1}} = d(x_p) + dp(y_p) / d(x_p)$ $\frac{\partial S_c}{\partial p_1} = d(I(p)) / d(p_1)$ $\frac{\partial S_c}{\partial y_p} = dp(x_p) / d(y_p) + d(y_p)$
				Deletion of a part of construct		
				Modifying the existing rule with few construct changes		